

Challenge Write-Up Template

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	Piece by Piece
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 11, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

Piece by Piece is a file-reassembly challenge rather than a cracking or exploitation one. The flag has been split into several chunks and placed in the home directory; the real task is to figure out the correct order to stitch those chunks back together into their original file, and then deal with whatever protection (in this case, a password) that file has before the flag can be read.

ping-cmd

General Skills **Easy**

by Yahaya Meddy · picoCTF 2026

Can you make the server reveal its secrets? It seems to be able to ping Google DNS, but what happens if you get a little creative with your input?

INSTANCE ⓘ

Launch Instance Additional details appear once you launch your instance.

HINTS ⓘ

- 1 The program uses a shell command behind the scenes.
- 2 Sometimes, You can run more than one command at a time.

FLAG ⓘ

Submit

debug info: [u:1109650 e: p: c:757 i:298645]

Bookmark 11,000 96%

Open in Workspace

2. Initial Analysis / Reconnaissance

The first move on any challenge like this is simply to log in and look around - `ls -la` in the home directory shows exactly what you're working with before you assume anything. Seeing a set of same-sized files named `part_aa`, `part_ab`, `part_ac`, `part_ad`, and `part_ae` (plus a smaller final one) is the classic naming pattern produced by the Unix `split` command, which strongly suggests the goal is to concatenate them back together in that same alphabetical order to recover the original file.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
<code>ssh -p 51284 ctf-player@dolphin-cove.picoctf.net</code>	Log into the challenge instance to access the home directory files	Shell access as <code>ctf-player@pico-chall</code>
<code>ls -la / cat instructions.txt</code>	See what files exist and read the challenge's own instructions for how they fit together	<code>instructions.txt</code> + five chunks (<code>part_aa</code> - <code>part_ae</code>); hint says: combine the parts, then unzip with password "supersecret"
<code>for f in part_*; do echo "==== \$f ====="; cat "\$f"; echo; done</code>	Preview each part's raw contents to confirm they're binary fragments of one file, in order	<code>part_aa</code> starts with the bytes 'PK', the standard ZIP file signature - confirming the parts are sequential pieces of a .zip
<code>cat part_aa part_ab part_ac part_ad part_ae > challenge.zip / file challenge.zip / unzip -l challenge.zip</code>	Reassemble the parts in order, then confirm the result is a valid zip and preview its contents before extracting	<code>challenge.zip</code> confirmed as 'Zip archive data'; contains one file, <code>flag.txt</code> (45 bytes)
<code>unzip -P supersecret challenge.zip && cat flag.txt</code>	Extract the archive using the password given in the instructions, then read the result	<code>picoCTF{z1p_and_spl1t_f1l3s_4r3_fun_da494d2e}</code>

4. Solution Walkthrough

Step 1

Connect to the challenge instance with the credentials given in the prompt: `ssh -p 51284 ctf-player@dolphin-cove.picoctf.net`, then log in with the provided password. Once in, you land in the `ctf-player` home directory, ready to look at whatever files the challenge left there.

```
kyxxajoy@PASCUA:~$ ssh -p 51284 ctf-player@dolphin-cove.picoctf.net
The authenticity of host '[dolphin-cove.picoctf.net]:51284 ([3.13.34.175]:51284)' can't be established.
ED25519 key fingerprint is: SHA256:rdvWhF7Klwlu1EivysxTh/FjeqFI0dSEK5gaelaW9
+8
```

Step 2

Before combining anything, check exactly what's in the directory with `ls -la`. Besides the usual dotfiles, there's an `instructions.txt` and five part files (`part_aa` through `part_ae`) of nearly identical size - a strong hint that these are sequential chunks of one larger file that was split apart, most likely with the `split` command, and need to be joined back together in order.

```
ctf-player@pico-chall$ ls -la
total 28
drwxr-xr-x 1 ctf-player ctf-player 20 Aug 17 05:27 .
drwxr-xr-x 1 root      root      24 Feb  4 2026 ..
drwx----- 2 ctf-player ctf-player 34 Aug 17 05:27 .cache
-rw-r--r-- 1 root      root      67 Feb  4 2026 .profile
-rw-r--r-- 1 ctf-player ctf-player 282 Feb  4 2026 instructions.txt
-rw-r--r-- 1 ctf-player ctf-player 51 Feb  4 2026 part_aa
-rw-r--r-- 1 ctf-player ctf-player 51 Feb  4 2026 part_ab
-rw-r--r-- 1 ctf-player ctf-player 51 Feb  4 2026 part_ac
-rw-r--r-- 1 ctf-player ctf-player 51 Feb  4 2026 part_ad
-rw-r--r-- 1 ctf-player ctf-player 35 Feb  4 2026 part_ae
ctf-player@pico-chall$ ls -lah
total 28K
drwxr-xr-x 1 ctf-player ctf-player 20 Aug 17 05:27 .
drwxr-xr-x 1 root      root      24 Feb  4 2026 ..
```

Step 3

Read `instructions.txt` before guessing at a workflow - challenges usually spell out exactly what's expected. Here it confirms the plan directly: the flag is inside a password-protected zip that's been split into the `part_*` files, Linux commands should be used to recombine them, and the zip's password is "supersecret". Previewing each part's raw bytes with a quick `for` loop is also useful: `part_aa`'s content begins with `PK`, the standard signature at the start of every ZIP file, which confirms these chunks are indeed sequential pieces of one .zip archive rather than something else.

```
ctf-player@pico-chall$ cat instructions.txt
Hint:

- The flag is split into multiple parts as a zipped file.
- Use Linux commands to combine the parts into one file.
- The zip file is password protected. Use this "supersecret" password to extract the zip file.
- After unzipping, check the extracted text file for the flag.
```

```
ctf-player@pico-chall$ for f in part_*; do echo "==== $f ====="; cat "$f";
echo; done
==== part_aa =====
PK
D\9flag.txtUT      fi
==== part_ab =====
ux
\mD#?L_^^l^x'Qs
==== part_ac =====
N6>c.#)P9-PK
♦
==== part_ad =====
D\9flag.txtUTfiux
```

Additional steps (add as needed)

With the order confirmed by the alphabetical part names, concatenate them in that same order using cat: `cat part_aa part_ab part_ac part_ad part_ae > challenge.zip`. Running `file challenge.zip` on the result confirms it's now valid zip archive data, and `unzip -l challenge.zip` lists its contents without extracting - here, a single 45-byte file called `flag.txt`. Since the instructions said the archive is password-protected, extract it with the given password: `unzip -P supersecret challenge.zip`. Finally, `cat flag.txt` reveals the flag: `picoCTF{z1p_and_spl1t_f1l3s_4r3_fun_da494d2e}`.

```
==== part_ae ====

PKN♦
ctf-player@pico-chall$ cat part_aa part_ab part_ac part_ad part_ae > challenge.zip
ctf-player@pico-chall$ file challenge.zip
challenge.zip: Zip archive data, at least v1.0 to extract
ctf-player@pico-chall$ file challenge.zip
challenge.zip: Zip archive data, at least v1.0 to extract

ctf-player@pico-chall$ file challenge.zip
challenge.zip: Zip archive data, at least v1.0 to extract
ctf-player@pico-chall$ unzip -l challenge.zip
Archive:  challenge.zip
  Length      Date    Time    Name
  -----
      45      2026-02-04  22:40    flag.txt
  -----
      45                          1 file
ctf-player@pico-chall$ unzip -P supersecret challenge.zip
Archive:  challenge.zip
 extracting: flag.txt
ctf-player@pico-chall$ cat flag.txt
picoCTF{z1p_and_spl1t_f1l3s_4r3_fun_da494d2e}ctf-player@pico-chall$
```

5. Flag

`picoCTF{z1p_and_spl1t_f1l3s_4r3_fun_da494d2e}`

6. Key Concept / What I Learned

The key concept is file splitting and reassembly: large or sensitive files are sometimes broken into smaller chunks (with tools like `split`) for transfer or storage, and reconstructing them is as simple as concatenating the pieces back together in the correct order with `cat`. Combined with recognizing file signatures (like PK for zip) from raw bytes, this is a useful, low-level skill for handling any challenge or forensics task where a file has been fragmented or its extension/type isn't obvious up front.

7. References

None beyond the challenge's own instructions.txt and standard documentation for `split`, `cat`, `file`, and `unzip`.
